

Bron CGGMP21

Security Assessment and Fix Review

October 23, 2026

Prepared for:
Alireza Rafiei
Bron Labs

Prepared by: **Joop van de Pol**

Table of Contents

Table of Contents **1**
Project Summary **2**
Executive Summary **3**
Project Goals **5**
Project Targets **6**
Project Coverage **7**
Summary of Findings **8**
Detailed Findings **9**
1. Deserialization of Paillier elements can panic due to nil dereference 9
2. Round 1 DKG commitment message does not separate Pedersen values 11
3. Deserialization failure on message receive does not include participant ID in error 13
4. Joye and Paillier safe prime generation reduces entropy of resulting safe primes 14
5. Modular integer element size reports bits instead of bytes 16
6. User is not sufficiently warned about thread-safe PRNG requirements for prime generation 18
7. CGGMP21 signing proofs are not bound to their proof slot 22
8. Discrete logarithm with ElGamal commitment proof verification does not bind witness to commitment 24
9. Red Alert rounds lack round-state checks 28
A. Vulnerability Categories **30**
B. Code Quality Issues **32**
C. Fix Review Results **39**
Detailed Fix Review Results 40
D. Fix Review Status Categories **42**
About Trail of Bits **43**
Notices and Remarks **44**

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Application Security and Cryptography
james.miller@trailofbits.com

The following consultant was associated with this project:

Joop van de Pol, Consultant
joop.vandepol@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
June 30, 2026	Pre-project kickoff call
July 13, 2026	Status update #1
July 20, 2026	Status update #2
July 29, 2026	Delivery of report draft and report readout meeting
August 28, 2026	Delivery of final comprehensive report
October 16, 2026	Completion of fix review
October 23, 2026	Delivery of final comprehensive report with fix review appendix

Executive Summary

Engagement Overview

Bron Labs engaged Trail of Bits to review the security of the CGGMP21 implementation in the `bron-crypto` library.

The `bron-crypto` library includes implementations for various threshold signature schemes and many of the underlying mathematical primitives required for these schemes. The CGGMP21 implementation follows the corresponding paper, including protocol implementations both for distributed key generation and signing.

One consultant conducted the review from July 6 to July 28, 2026, for a total of three engineer-weeks of effort. Our testing efforts focused on the distributed key generation (DKG) and signing protocols with all underlying zero-knowledge (ZK) proofs. With full access to source code and documentation, we performed static and dynamic testing of the CGGMP21 implementation, using automated and manual processes.

As part of this project, we also reviewed the library for issues related to misuse resistance and denial of service (DoS). However, Bron Labs considers DoS attacks out of scope for its library due to its intended usage. All findings related to such attacks have been classified as low severity as a result, except where they enable other attacks.

Observations and Impact

The library implements many of the required checks in the deserialization routines, which prevents a wide range of attacks from malicious participants sending incorrect values. Nonetheless, we found a few missing checks that could cause a panic during DKG and signing (TOB-BLCG-1). One ZK proof construction deviates from the description in the paper, enabling a rushing adversary for the nonce during signing (TOB-BLCG-8). The remaining issues focus on general hardening recommendations.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Bron Labs take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Deduplicate the code.** Currently, the codebase includes significant code duplication among helper functions for ZK proofs. This should be refactored to eliminate duplicate code in order to improve maintainability.
- **Instruct users on how to support identifiable abort.** The CGGMP21 signing protocol uses both broadcast and peer-to-peer (P2P) communication, and it includes

identifiable abort. However, the paper states that their security analysis assumes that the P2P communication is implemented using an authenticated broadcast, in order to make identifiable abort easier, because participants cannot claim that they received something else via P2P communication at a later point in time. While the specific communication layer was not in scope for this engagement, the library should document for users how they can implement the broadcast and P2P communication in such a way that identifiable abort is possible.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	0
Medium	0
Low	2
Informational	5
Undetermined	2

CATEGORY BREAKDOWN

Category	*Count*
Cryptography	4
Data Validation	2
Denial of Service	2
Timing	1

Project Goals

The engagement was scoped to provide a security assessment of the Bron Labs CGGMP21 implementation. Specifically, we sought to answer the following non-exhaustive list of questions:

- Does the CGGMP21 threshold signing implementation follow the specification, including all required checks and ZK proofs?
- If there are deviations from the specification, do these deviations affect the security of the scheme?
- Are the ZK proofs used in CGGMP21 properly implemented to prevent information leakage about private key shares?
- Are the commitments properly implemented to protect against rushing adversaries?
- Does the system handle malformed or malicious inputs during the signature generation process?
- How does the implementation ensure that threshold requirements are properly enforced throughout the protocol execution?

Project Targets

The engagement involved reviewing and testing the following target.

bron-crypto (CGGMP21)

Repository	https://github.com/bronlabs/bron-crypto/
Version	`d2373cf0578cee18505a9051cfc50fda671c3670`

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Review of the following components using manual and automated methods:
  - CGGMP21 auxiliary parameter generation during the DKG
  - CGGMP21 signing protocol
  - All underlying ZK proofs used by CGGMP21
  - Joye–Paillier prime and safe prime generation (including the corresponding paper)

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. During this project, we were unable to perform comprehensive testing of the following system elements, which may warrant further review:

- The base DKG (Canetti) implementation
- The Monotone Span Program implementation
- The zero-resharing protocol (HJKY95) implementation
- Third-party dependencies

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Deserialization of Paillier elements can panic due to nil dereference	Denial of Service	**Low**
2	Round 1 DKG commitment message does not separate Pedersen values	Cryptography	**Informational**
3	Deserialization failure on message receive does not include participant ID in error	Denial of Service	**Low**
4	Joye and Paillier safe prime generation reduces entropy of resulting safe primes	Cryptography	**Informational**
5	Modular integer element size reports bits instead of bytes	Data Validation	**Informational**
6	User is not sufficiently warned about thread-safe PRNG requirements for prime generation	Timing	**Undetermined**
7	CGGMP21 signing proofs are not bound to their proof slot	Cryptography	**Informational**
8	Discrete logarithm with ElGamal commitment proof verification does not bind witness to commitment	Cryptography	**Undetermined**
9	Red Alert rounds lack round-state checks	Data Validation	**Informational**

Detailed Findings

1. Deserialization of Paillier elements can panic due to nil dereference

Severity: **Low**	Difficulty: **Medium**
Type: Denial of Service	Finding ID: TOB-BLCG-1
Target: `pkg/base/nt/znstar/cbor.go`	

Description

The deserialization of a simple `znstar` modulus performs a square operation on this modulus before checking that it is not `nil`. The CBOR decoder will convert missing fields to `nil` values, so omitting the field (or explicitly encoding a `null` value) will cause a crash in the recipient.

```
case *modular.SimpleModulus:
	dto, err := serde.UnmarshalCBOR[paillierGroupUnknownOrderDTO](data)
	if err != nil {
		return errs.Wrap(err)
	}
	n2 := dto.N.Square()
```

Figure 1.1: Deserialization panics if `N` is `nil`. (pkg/base/nt/znstar/cbor.go#L166–L171)

As an aside, there are no checks on the length of the modulus. Participants could send arbitrarily large moduli, wasting resources when the recipient squares them and for all subsequent calculations.

The same issue arises when deserializing the ring Pedersen commitment key during the round 2 broadcast (or during the round 3 PRM proof, round 4 Blum proof, and round 4 FAC proof). A malformed nested RSA group element with a missing or `nil` `V` value causes a panic when deserialization tries to access the modulus, as shown in figure 1.2.

```
case *modular.SimpleModulus:
	dto, err := serde.UnmarshalCBOR[rsaGroupUnknownOrderElementDTO](data)
	if err != nil {
		return errs.Wrap(err)
	}
	g, err := NewRSAGroupOfUnknownOrder(dto.V.Modulus())
	if err != nil {
		return errs.Wrap(err)
	}
```

Figure 1.2: Deserialization panics if V is `nil`. (pkg/base/nt/znstar/cbor.go#L431–L439)

Exploit Scenario

A malicious participant sends a round 2 broadcast message with a missing Paillier modulus field, which crashes all recipients of this message, causing a denial of service.

Recommendations

Short term, verify that the Paillier modulus is not nil before using it.

Long term, consider adding specific `dto.Validate` functions that verify each DTO requiring custom deserialization. Add negative testing for deserialization with missing fields or explicitly encoded `nil` values.

2. Round 1 DKG commitment message does not separate Pedersen values

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-BLCG-2
Target: `pkg/mpc/signatures/ecdsa/cggmp21/keygen/dkg/messages.go`	

Description

The canonical encoding of the round 1 DKG commitment message does not properly separate the `s` and `t` values, which can lead to an equivocation. Figure 2.1 shows that values `s` and `t` do not get length prefixes in the component, as opposed to other elements that are not of fixed size.

This is not exploitable because the proof `Psi` also includes the values `s` and `t`, and its proof transcript does prefix these values with their lengths.

```
// Bytes is the canonical encoding hashed by the round-1 commitment and
// re-derived by the verifier when opening it in round 3; the commitment's
// binding rests on it being injective.
func (m *CommitmentMessage[P, B, S]) Bytes() []byte {
	var data []byte
	data = append(data, m.SessionID[:]...)
	data = binary.LittleEndian.AppendUint64(data, uint64(m.SharingID))
	data = binary.LittleEndian.AppendUint64(data,
uint64(m.PaillierPublicKey.Group().N().TrueLen()))
	data = append(data, m.PaillierPublicKey.Group().N().BytesBE()...)
	data = binary.LittleEndian.AppendUint64(data,
uint64(m.RingPedersenCommitmentKey.Group().Modulus().TrueLen()))
	data = append(data,
m.RingPedersenCommitmentKey.Group().Modulus().BytesBE()...)
	data = append(data, m.RingPedersenCommitmentKey.S().Value().BytesBE()...)
	data = append(data, m.RingPedersenCommitmentKey.T().Value().BytesBE()...)
	data = binary.LittleEndian.AppendUint64(data, uint64(len(m.Psi)))
	data = append(data, m.Psi...)
	data = binary.LittleEndian.AppendUint64(data, uint64(len(m.Rid)))
	data = append(data, m.Rid...)
	return data
}
```

Figure 2.1: The `s` and `t` values are appended to the encoding without a length prefix. (pkg/mpc/signatures/ecdsa/cggmp21/keygen/dkg/messages.go#L53–L71)

Recommendations

Short term, prefix the `s` and `t` values with their lengths.

Long term, consider creating special append functions (e.g., one for appending fixed-size elements and one for appending variable-size elements).

3. Deserialization failure on message receive does not include participant ID in error

Severity: **Low**	Difficulty: **Medium**
Type: Denial of Service	Finding ID: TOB-BLCG-3
Target: `pkg/network/router.go`, `pkg/network/exchange.go`	

Description

When receiving messages, a failed deserialization triggers an error that does not include the identifiable abort tag corresponding to the sender. This goes both for the network router (figure 3.1) and the unicast exchange (figure 3.2) implementations.

```
message, err := serde.UnmarshalCBOR[routerMessage](serializedMessage)
if err != nil {
	return nil, errs.Wrap(err).WithMessage("failed to decode message")
}
```

Figure 3.1: Deserialization error does not include the identifiable abort tag. (pkg/network/router.go#L106–L109)

```
msg, err := serde.UnmarshalCBOR[U](m)
if err != nil {
	return nil, errs.Wrap(err).WithMessage("failed to unmarshal message")
}
```

Figure 3.2: Deserialization error does not include the identifiable abort tag. (pkg/network/exchange.go#L39–L42)

At worst, this will cause a denial of service, as the protocol stops in a way that cannot be attributed to any party.

Exploit Scenario

A malicious participant wants to abort the protocol without being identified. They send a badly serialized message to another party, and that party aborts the protocol without identifying the malicious party.

Recommendations

Short term, add the corresponding identifiable abort tag to deserialization errors.

Long term, document for each error why it should or should not be identifiable.

4. Joye and Paillier safe prime generation reduces entropy of resulting safe primes

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-BLCG-4
Target: `pkg/base/nt/primes.go`	

Description

The safe prime generation method uses a variant described by Joye and Paillier, which forces each prime candidate to be a quadratic non-residue modulo a set of small odd primes. This reduces the resulting entropy of the primes by approximately 1 bit per odd prime, as it rules out half of the possible residues modulo each odd prime.

```
//  2. (q-1)/2 is coprime to Π. From (1) q is a QNR mod every odd p_i | Π,
//     hence q ≠ 1 (mod p_i), hence p_i ∤ (q-1), hence p_i ∤ (q-1)/2.
```

Figure 4.1: The prime candidate is a quadratic non-residue modulo each small prime in the prime sieve. (pkg/base/nt/primes.go#L226–L227)

For 1536-bit primes, the entropy loss per prime is approximately 178 bits. This is not sufficient to exploit using known attacks, and the structure of the information does not lend itself to the usual Coppersmith-type attacks that are used for these kinds of problems.

Specifically, the entropy loss is calculated by comparing how many residue classes remain possible modulo each small odd prime in the sieve. For a small prime `p_i`, an ordinary safe prime `q = 2r + 1` cannot satisfy `q ≡ 0 mod p_i`, because then `q` would be divisible by `p_i`, and it also cannot satisfy `q ≡ 1 mod p_i`, because then `p_i` would divide `(q - 1) / 2 = r`. That leaves `p_i - 2` plausible residue classes for normal safe primes. The Joye–Paillier variant further requires `q` to be a quadratic non-residue modulo `p_i`, which allows only `(p_i - 1) / 2` residue classes. Therefore, the retained fraction for that prime is `((p_i - 1) / 2) / (p_i - 2)`, and the entropy loss is `log2((p_i - 2) / ((p_i - 1) / 2)) = log2(2(p_i - 2)/(p_i - 1))` bits. This value approaches one bit for large `p_i`, so the rough estimate is “about one bit per small odd prime.”

In the implementation, the 1536-bit setup selects 181 small odd primes, and summing this expression over those primes gives about 177.8 bits of entropy loss per generated safe prime. This does not mean that 178 contiguous bits of the prime are known; rather, the prime is known to satisfy many Legendre-symbol constraints, one modulo each small prime in Π. The remaining candidates are spread across many Chinese Remainder Theorem

(CRT) residue classes, which is why this kind of entropy loss is much less directly exploitable than knowing high bits, low bits, or a single strong congruence.

Recommendations

Short term, consider using an alternative efficient safe prime generation algorithm that only restricts the residues of the prime candidate to be greater than one (e.g., based on the subsequent work by Clavier et al.).

Long term, document whether the entropy loss is worth the efficiency gains.

5. Modular integer element size reports bits instead of bytes

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Data Validation	Finding ID: TOB-BLCG-5
Target: `pkg/base/types.go`, `pkg/base/nt/num/zn.go`, `pkg/base/nt/numct/nat.go`	

Description

The modular integer type reports its element size in bits, whereas the `ElementSize` interface expects bytes.

```
// If elements are atomic, ElementSize returns the **exact** number of bytes
(implementation-dependent) required to represent an element.
// If elements are collections of atomic elements, ElementSize returns the size of
an individual element.
// If elements are variable size, ElementSize returns -1.
ElementSize() int
```

Figure 5.1: The interface reports the byte size of elements. (pkg/base/types.go#L42–L45)

However, for the modular integer type `ZMod`, the implemented `ElementSize` calls `AnnouncedLen`, which returns the number of bits, as shown in figure 5.2 and figure 5.3.

```
// ElementSize returns the size in bytes of an element.
func (zn *ZMod) ElementSize() int {
	return zn.n.AnnouncedLen()
}

// WideElementSize returns the size in bytes of a wide element.
func (zn *ZMod) WideElementSize() int {
	return 2 * zn.ElementSize()
}
```

Figure 5.2: The interface implementations for modular integers call `AnnouncedLen`. (pkg/base/nt/num/zn.go#L97–L105)

```
// AnnouncedLen returns the announced length in bits of n. Safe to be used publicly.
func (n *Nat) AnnouncedLen() int {
	return ((*saferith.Nat)(n).AnnouncedLen())
}
```

Figure 5.3: The function returns the size in bits. (pkg/base/nt/numct/nat.go#L273–L276)

This is not exploitable, but it does cause a mismatch when hashing to a `ZMod` element (e.g., in the CGGMP21 Blum modulus proof).

Recommendations

Short term, update the function to return byte size, not bit size.

Long term, add unit tests to determine that the element size functions return the expected sizes.

6. User is not sufficiently warned about thread-safe PRNG requirements for prime generation

Severity: **Undetermined**	Difficulty: **Undetermined**
Type: Timing	Finding ID: TOB-BLCG-6
Target: `pkg/base/nt/primes.go`, `pkg/mpc/signatures/ecdsa/cggmp21/keygen/dkg/participant.go`, `pkg/mpc/signatures/ecdsa/cggmp21/keygen/dkg/rounds.go`	

Description

The prime generation methods create goroutines that concurrently access the user-provided pseudorandom number generator (PRNG) object, without warning the user that the PRNG should be thread-safe. The `generatePrimePair` function launches parallel prime searches for p and q, as shown in figure 6.1.

```
g.Go(func() error {
	var err error
	for {
		pCandidate, err = gen(set, primeBits, prng)
		if err != nil {
			return errs.Wrap(err).WithMessage("cannot generate prime")
		}
		pCandidateNat, err = num.NPlus().FromCardinal(pCandidate.Cardinal())
		if err != nil {
			return errs.Wrap(err).WithMessage("cannot convert p to NatPlus")
		}
		if pCandidateNat.Square().TrueLen() >= int(keyLen) { //
TrueLen()/BitLen() gives the index of the highest set bit plus one, so it's
int(keyLen) not int(keyLen-1)
			return nil
		}
	}
})
g.Go(func() error {
	var err error
	for {
		qCandidate, err = gen(set, primeBits, prng)
		if err != nil {
			return errs.Wrap(err).WithMessage("cannot generate prime")
		}
		qCandidateNat, err = num.NPlus().FromCardinal(qCandidate.Cardinal())
		if err != nil {
			return errs.Wrap(err).WithMessage("cannot convert q to NatPlus")
		}
```

```
                if qCandidateNat.Square().TrueLen() >= int(keyLen) {
                        return nil
                }
        }
})
if err := g.Wait(); err != nil {
        return nilN, nilN, errs.Wrap(err).WithMessage("cannot generate primes")
}
```

Figure 6.1: pkg/base/nt/primes.go#L541–L575

Each of these goroutines calls the gen function, which, when instantiated with either `GenerateBlumPrime` or `GenerateSafePrime`, will perform one or more reads on the `prng` object. The function `GenerateSafePrime` itself also launches a parallel search across multiple workers, as shown in figure 6.2. While the `GenerateSafePrime` documentation states that it requires a thread-safe PRNG, this is a low-level internal function that likely will not be used directly by a user.

```
// ———————————————————————————————————————————————————————————————————
// Parallel scaffolding.
// ———————————————————————————————————————————————————————————————————
// One worker per CPU. Each runs an independent (x, orbit) per Fig. 6.
// First worker to find a safe prime wins; cancel() stops the rest.
workers := runtime.GOMAXPROCS(0)
ctx, cancel := context.WithCancel(context.Background())
defer cancel()
g, ctx := errgroup.WithContext(ctx)
results := make(chan E, 1)
defer close(results)
```

Figure 6.2: pkg/base/nt/primes.go#L279–L289

Instead, the user will use the DKG function `NewParticipant` to create a participant, passing it a PRNG object. Its documentation notes that it should be cryptographically secure, but does not mention thread safety, as shown in figure 6.3.

```
// NewParticipant binds a participant to a session context, the party's existing
// secret-share base shard, and an explicit randomness source. It checks that the
// context's holder identity and quorum match the base shard, derives the
// hash-commitment key from the session transcript (domain-separated per this
// protocol), and instantiates the Fiat-Shamir compilers for the Π_prm and Π_mod
// proofs. The base shard's public key must live over an ECDSA curve. prng must
// be a cryptographically secure source; the Paillier/ring-Pedersen key
// generation in Round1 draws from it.
func NewParticipant[P curves.Point[P, B, S], B algebra.PrimeFieldElement[B], S
algebra.PrimeFieldElement[S]](ctx *session.Context, baseShard *mpc.BaseShard[P, S],
prng io.Reader) (*Participant[P, B, S], error) {
```

Figure 6.3: pkg/mpc/signatures/ecdsa/cggmp21/keygen/dkg/participant.go#L74–L82

When running the first DKG round, each participant also parallelizes the search for the Paillier key (requiring Blum primes) and the ring Pedersen key (requiring safe primes), as shown in figure 6.4.

```
eg := errgroup.Group{}
// step 1(a)
eg.Go(func() error {
	var err error
	p.state.paillierSecretKey, err =
paillier.SampleBlumSecretKey(base.IFCKeyLength, p.prng)
	if err != nil {
		return errs.Wrap(err).WithMessage("cannot sample Paillier secret key")
	}
	return nil
})
// step 1(b)
eg.Go(func() error {
	var err error
	p.state.ringPedersenSecretKey, err =
intcom.SampleTrapdoorKey(base.IFCKeyLength, p.prng)
	if err != nil {
		return errs.Wrap(err).WithMessage("cannot sample ring-Pedersen trapdoor
key")
	}
	proverCtx := p.ctx.Clone()
	proverCtx.Transcript().AppendBytes(proverIDLabel, p.ctx.HolderID().Bytes())

	for id := range p.ctx.OtherPartiesOrdered() {
		verifierCtx := p.ctx.Clone()
		verifierCtx.Transcript().AppendBytes(proverIDLabel, id.Bytes())
		p.state.prmVerifierCtx[id] = verifierCtx
	}

	prmProver, err := p.state.prmfs.NewProver(proverCtx)
	if err != nil {
		return errs.Wrap(err).WithMessage("cannot create PRM prover")
	}
	prmStatement, err := prm.NewStatement(p.state.ringPedersenSecretKey.Export())
	if err != nil {
		return errs.Wrap(err).WithMessage("cannot create PRM statement")
	}
	prmWitness, err := prm.NewWitness(p.state.ringPedersenSecretKey)
	if err != nil {
		return errs.Wrap(err).WithMessage("cannot create PRM witness")
	}
	p.state.psiI, err = prmProver.Prove(prmStatement, prmWitness)
	if err != nil {
		return errs.Wrap(err).WithMessage("cannot generate PRM proof")
	}
	return nil
})
if err := eg.Wait(); err != nil {
```

```
        return nil, errs.Wrap(err).WithMessage("round 1 failed")
}
```

Figure 6.4: pkg/mpc/signatures/ecdsa/cggmp21/keygen/dkg/rounds.go#L35–L81

The library provides several thread-unsafe PRNGs (i.e., the `pcg`, `fkechacha20`, and `nist` packages). These modify internal state without a mutex, leading to data races between concurrent reads. Multiple goroutines could read overlapping entropy streams or corrupted (e.g., zeroed out) internal state.

Exploit Scenario

An honest participant runs the protocol using a thread-unsafe PRNG. The concurrent reads either corrupt or overlap the entropy streams used for one or more of the prime candidates. In the worst case, this could lead to exploitable patterns in the prime candidates that lead to prime recovery.

Recommendations

Short term, wrap the user-provided `prng` in a `csprng.NewThreadSafePrng` when running the DKG. Document the user requirements for the PRNG.

Long term, add race tests for PRNG use in key generation. Consider defining an explicit concurrency-safe entropy interface for use with multiple goroutines.

7. CGGMP21 signing proofs are not bound to their proof slot

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-BLCG-7
Target: `pkg/mpc/signatures/ecdsa/cggmp21/signing/rounds.go`	

Description

The two pairs of `enc-elg` and `aff-g` ZK proofs use the same transcript within each pair, which means that it is possible to replay the same proof twice. The implementation uses proof helpers `proveEncElg` and `proveAffG`, which are called shortly after each other during the corresponding signing round, as shown in the following code for `enc-elg` (figure 7.1).

```
// step 1.c
psi0 := make(map[sharing.ID]compiler.NIZKPoKProof)
psi1 := make(map[sharing.ID]compiler.NIZKPoKProof)
for j := range s.ctx.OtherPartiesOrdered() {
	proof0, err := s.proveEncElg(j, k, bigK, rho, a, bigY, bigA)
	if err != nil {
		return nil, nil, errs.Wrap(err).WithMessage("cannot prove enc-elg for k
to %d", j)
	}
	proof1, err := s.proveEncElg(j, gamma, bigG, nu, b, bigY, bigB)
	if err != nil {
		return nil, nil, errs.Wrap(err).WithMessage("cannot prove enc-elg for
gamma to %d", j)
	}

	psi0[j] = proof0
	psi1[j] = proof1
}
```

Figure 7.1: The proofs are created together during the same protocol step. (pkg/mpc/signatures/ecdsa/cggmp21/signing/rounds.go#L105–L120)

These helper functions derive a subcontext using the proof sender and recipient and append the prover ID to the transcript, as shown in figure 7.2.

```
proverCtx, err := s.ctx.SubContext(hashset.NewComparable(s.ctx.HolderID(),
recipient).Freeze())
if err != nil {
	return nil, errs.Wrap(err).WithMessage("cannot create subcontext")
}
proverCtx.Transcript().AppendBytes(proverID, s.shard.Share().ID().Bytes())
```

```
prover, err := encElg.NewProver(proverCtx)
if err != nil {
	return nil, errs.Wrap(err).WithMessage("cannot create enc-elg prover")
}
```

Figure 7.2: The subcontext is derived from the holder and recipient and the transcript is updated based on prover ID.
(pkg/mpc/signatures/ecdsa/cggmp21/signing/rounds.go#L636–L644)

As a result, both pairs of `enc-elg` proofs start from the same transcript, which means that the prover can replay their own proof, as long as they ensure that the statements are the same.

Several considerations mitigate the impact of this issue: the issue does not give a malicious prover any advantage, and it leaks information on their private signing share. Furthermore, the paper does not mandate proof separation for these proofs. Nevertheless, we advise adding a slot label as a defense-in-depth measure.

Recommendations

Short term, consider adding slot labels to the `enc-elg` and `aff-g` proofs for signing.

Long term, consider defining distinct proof slot labels for every individual ZK proof.

8. Discrete logarithm with ElGamal commitment proof verification does not bind witness to commitment

Severity: **Undetermined**	Difficulty: **Undetermined**
Type: Cryptography	Finding ID: TOB-BLCG-8

Target: `pkg/proofs/elgamal/elog/README.md`, `pkg/proofs/elgamal/elog/protocol.go`, `pkg/proofs/sigma/compose/sigand/cartesian.go`

Description

The `elog` proof verification does not bind the witness to the ElGamal commitment, relying instead on honest prover behavior to bind them. As a result, a malicious prover can change their committed value after observing decommitted values of other parties.

The `elog` proof is implemented as a Cartesian product of two proofs, as described in its README (figure 8.1).

```
## Implementation

Rather than instantiating Figure 23 as a single Maurer homomorphism, this
package realizes $R_{\mathrm{elog}}$ as the AND-composition of two existing
Maurer protocols, glued by a consistency check on the witnesses:

1. **`elcomop`** ([../elcomop](../elcomop)) — proves knowledge of an opening
   $(M', \lambda) \in G \times \mathbb{F}_q$ of the ElGamal commitment
   $(L, M) = (g^\lambda,\; M' \cdot X^\lambda)$.
2. **`schnorr`** ([../../dlog/schnorr](../../dlog/schnorr)) instantiated with
   base $h$ — proves knowledge of $y \in \mathbb{F}_q$ such that $Y = h^y$.

The AND-composition runs both sub-protocols under the same verifier
challenge $e$. On its own this would only prove
"$\exists\, (M', \lambda, y)$ with $L = g^\lambda,\ M = M' X^\lambda,\ Y = h^y$";
`NewWitness` therefore enforces the binding

$$
M' = g^y
$$

on the composed witness before any transcript is produced. Substituting
$M' = g^y$ into the elcomop relation recovers $M = g^y X^\lambda$, exactly the
$R_{\mathrm{elog}}$ requirement.
```

Figure 8.1: The `elog` proof README describes that the proofs themselves are not sufficient and rely on an honest witness construction function. (pkg/proofs/elgamal/elog/README.md#L58–L81)

As stated, the binding is enforced by `NewWitness` (figure 8.2), a method used by honest provers to generate the proof. This does not in any way influence the verifier, which, as a result, does not check the binding. The Cartesian product construction enforces only that the proofs use the same challenge bytes to construct their challenges (figure 8.3).

```
// NewWitness composes an elcomop witness w1 = (M', lambda) with a Schnorr
// witness w2 = y into an elog witness. It enforces the elog binding M' = g^y,
// where g is the generator of the group the elcomop plaintext lives in, so
// that the resulting elcomop ciphertext (L, M) satisfies M = g^y * X^lambda.
func NewWitness[G algebra.PrimeGroupElement[G, S], S
algebra.PrimeFieldElement[S]](w1 *elcomop.Witness[G, S], w2 *schnorrpok.Witness[S])
(*Witness[G, S], error) {
	if w1 == nil || w2 == nil {
		return nil, proofs.ErrInvalidArgument.WithMessage("witnesses cannot be
nil")
	}
	message, _ := w1.Value().Components()
	group := algebra.StructureMustBeAs[algebra.PrimeGroup[G,
S]](message.Structure())
	if !message.Equal(group.ScalarBaseOp(w2.W)) {
		return nil, proofs.ErrInvalidArgument.WithMessage("invalid witness:
Schnorr witness does not match ElGamal commitment witness")
	}
	out, err := sigand.CartesianComposeWitnesses(w1, w2)
	if err != nil {
		return nil, errs.Wrap(err).WithMessage("cannot compose witnesses")
	}
	return out, nil
}
```

Figure 8.2: The honest witness construction function enforces equality of witnesses. (pkg/proofs/elgamal/elog/protocol.go#L40–L58)

```
// Verify checks that the binary AND proof is valid.
//
// Both branch transcripts are verified using the same challenge.
func (p *ProtocolCartesian[X0, X1, W0, W1, A0, A1, S0, S1, Z0, Z1]) Verify(statement
*StatementCartesian[X0, X1], commitment *CommitmentCartesian[A0, A1], challengeBytes
sigma.ChallengeBytes, response *ResponseCartesian[Z0, Z1]) error {
	if err := p.sigma0.Verify(statement.X0, commitment.A0,
challengeBytes[:p.sigma0.GetChallengeBytesLength()], response.Z0); err != nil {
		return errs.Wrap(err).WithMessage("verification failed")
	}
	if err := p.sigma1.Verify(statement.X1, commitment.A1,
challengeBytes[:p.sigma1.GetChallengeBytesLength()], response.Z1); err != nil {
		return errs.Wrap(err).WithMessage("verification failed")
	}
```

```
        return nil
}
```

Figure 8.3: Both verification functions use the same challenge bytes. (pkg/proofs/sigma/compose/sigand/cartesian.go#L201–L213)

Each participant generates such a proof for `Gamma`, their contribution to the nonce, and for `Delta`, their check value to confirm correct multiplication of the nonce. This issue allows a malicious participant to change the value used to construct `Gamma` and `Delta`, disconnecting them from earlier commitments.

Changing the committed value for `Delta` is not useful, because the committed value `BigA` is tied to `BigK` using an earlier proof, and `BigK` is used in the multiplicative-to-additive (MtA) procedures. So, any change to `Delta` either results in a red alert or requires the malicious participant to guess the honest contributions in order to pass the later checks. Note that the malicious participant can trigger the red alert in such a way that they will be able to pass all checks.

Changing the committed value for `Gamma` allows a malicious participant to choose their ECDSA nonce contribution after seeing the public contribution values of other participants. It is not clear that this can be exploited, because the `elog` proof still enforces proof of knowledge of the discrete logarithm of the contribution, which prevents a rogue-nonce attack. For Schnorr signatures, such an issue would enable ROS-style attacks, but it is not clear how to perform such attacks on ECDSA, because of the non-algebraic transformation that maps the public nonce point to its x-coordinate to obtain the first signature component `r`.

Still, the security proofs of the CGGMP21 protocol will no longer apply, and there may be other attacks that can exploit this.

Exploit Scenario

A malicious participant broadcasts an invalid `BigDelta` with a verifying `elog` proof. They keep the `Delta` scalar consistent with their actual MtA transcript. Round 4 fails and triggers a red alert, which checks `delta_i`, `BigGamma_i`, `BigK_i`, and the `D` and `F` evidence from the MtA procedure. Since it does not check `BigDelta_i`, the participant is not caught.

Recommendations

Short term, replace the Cartesian product construction by a faithful implementation of figure 23 in the CGGMP21 paper. Specifically, the proof should be a single Maurer homomorphism with specific types for the 2-dimensional witness and 3-dimensional statement vectors.

Long term, implement ZK proofs from a single declarative relation unless the composition system supports shared witness variables. Add a note to your agent instructions that cryptographic check enforcement cannot live in prover functions.

9. Red Alert rounds lack round-state checks

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Data Validation	Finding ID: TOB-BLCG-9
Target: `pkg/mpc/signatures/ecdsa/cggmp21/signing/redalert.go`	

Description

The Red Alert implementation does not ensure that the round functions are executed in the correct order. The protocol implementations in the library keep track of the current round, and each round function verifies that the current state matches the round to be executed, before executing the round and incrementing the round in the current state. However, the Red Alert participant does not keep track of its own round. The first Red Alert round function (figure 9.1) only verifies that the underlying `cosigner` is in the correct round state.

```
// Round1 broadcasts the local red-alert openings and their NIZK proofs.
func (p *RedAlertParticipant[P, B, S]) Round1() (*RedAlertBroadcast[P, B, S], error)
{
	if p.cosigner().state.round != 4 {
		return nil, cggmp21.ErrInvalidRound.WithMessage("actual=%d
expected=%d", p.cosigner().state.round, 4)
	}

	// step 1
```

Figure 9.1: The first Red Alert round verifies only the round state of the underlying cosigner. (pkg/mpc/signatures/ecdsa/cggmp21/signing/redalert.go#L173–L179)

The second Red Alert round function (figure 9.2) does not verify that the first round has been executed, which can cause a panic because its `localBroadcast` will be `nil`. Additionally, it is possible to keep executing the second round.

```
// Round2 verifies all red-alert broadcasts and returns an identifiable abort on the
first bad proof.
func (p *RedAlertParticipant[P, B, S]) Round2(r1b
network.RoundMessages[*RedAlertBroadcast[P, B, S], *RedAlertParticipant[P, B, S]])
error {
	if err := network.ValidateIncomingMessages(p,
p.cosigner().ctx.OtherPartiesOrdered(), r1b); err != nil {
		return errs.Wrap(err).WithMessage("invalid red alert broadcasts")
	}

	messages := map[sharing.ID]*RedAlertBroadcast[P, B, S]{
```

```
            p.cosigner().ctx.HolderID(): p.localBroadcast,
        }
        for id := range p.cosigner().ctx.OtherPartiesOrdered() {
            msg, _ := r1b.Get(id)
            messages[id] = msg
        }
```

Figure 9.2: The second Red Alert round does not verify that the first round was executed. (pkg/mpc/signatures/ecdsa/cggmp21/signing/redalert.go#L212–L224)

Recommendations

Short term, add round state tracking to the `RedAlertParticipant` struct and update the round functions to track the round numbers.

Long term, add negative testing for executing rounds out of order and for repeating rounds.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Issues

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Blum modulus proof uses modulus bytes instead of statement bytes.** The `blummod` proof uses the modulus bytes as the info field for HKDF, instead of using the full statement bytes. This should be equivalent because the modulus is the only field in the statement, but using `statement.Bytes()` would be more idiomatic.

```
info = sliceutils.AppendLengthPrefixed(info,
statement.publicKey.Group().N().Bytes())
```

Figure B.1: The info field includes the modulus bytes instead of the statement bytes. (pkg/proofs/cggmp21/blummod/protocol.go#L314–L314)

- **Duplicate code.** There are many instances of duplicate code, especially among protocol helpers. Some examples are `validateCommitmentKey` for Pedersen commitment keys and `intSampleRangeSymmetricBits`. The latter appears verbatim in four different proof helpers, whereas the former appears in the PRM proof and the no small factors proof, with a difference in code comments. These helpers should be replaced by single exported functions that are used at all the call sites. These are merely a few examples; there are more instances in the implementation.

```
func validateCommitmentKey(commitmentKey *intcom.CommitmentKey) error {
	if commitmentKey == nil {
		return proofs.ErrInvalidArgument.WithMessage("commitment key
must not be nil")
	}
	s := commitmentKey.S()
	t := commitmentKey.T()
	if s == nil || t == nil {
		return proofs.ErrInvalidArgument.WithMessage("s and t must not
be nil")
	}
```

Figure B.2: Beginning of the `validateCommitmentKey` function (pkg/proofs/prm/helpers.go#L12–L20)

```
func validateCommitmentKey(commitmentKey *intcom.CommitmentKey) error {
	if commitmentKey == nil {
		return ErrInvalidArgument.WithMessage("commitment key must not
be nil")
	}
```

```
	s := commitmentKey.S()
	t := commitmentKey.T()
	if s == nil || t == nil {
		return ErrInvalidArgument.WithMessage("s and t must not be nil")
	}
```

Figure B.3: Same beginning of the `validateCommitmentKey` function in a different file (pkg/proofs/cggmp21/fac/protocol.go#L529–L537)

```
func intSampleRangeSymmetricBits(bitLen int, prngReader io.Reader) (*num.Int,
error) {
	buf := make([]byte, bitLen/8)
	if _, err := io.ReadFull(prngReader, buf); err != nil {
		return nil, errs.Wrap(err).WithMessage("could not sample integer
bytes")
	}

	out, err := num.Z().FromTwosComplementBytesBE(buf)
	if err != nil {
		return nil, errs.Wrap(err).WithMessage("could not parse sampled
integer")
	}
	return out, nil
}
```

Figure B.4: pkg/proofs/cggmp21/affg/helpers.go#L27–L38

- **Comment states the wrong range.** The comment for the function `Normalise` states a half-open range, but the stated range is incorrect. The open end should be on the other side, as shown in the underlying `ModSymmetric` function.

```
// Normalise returns m as a signed integer in the symmetric range (-N/2, N/2],
the
// canonical signed representative of the residue.
func (pt *Plaintext) Normalise() *num.Int {
	out, err := num.Z().FromUintSymmetric(pt.p)
	if err != nil {
		panic(err)
	}
	return out
}
```

Figure B.5: `Normalise` states that the range is `(-N/2, N/2]`. (pkg/encryption/paillier/paillier.go#L287–L295)

```
// ModSymmetric sets out = x mod m in the symmetric range [-m/2, m/2).
func (m *ModulusBasic) ModSymmetric(out *Int, x *Nat) {
	(*saferith.Int)(out).SetModSymmetric((*saferith.Nat)(x),
(*saferith.Modulus)(m))
}
```

Figure B.6: `ModSymmetric` states that the range is `[-m/2, m/2)`.
(pkg/base/nt/numct/modulus.go#L66–L69)

- **`aff-g` parameters are bigger than required by the paper.** The paper states that `delta` and `mu` are sampled using the smaller value `l` instead of `lPrime`. This is not a security issue but reduces the efficiency.

```
delta, err := intSampleRangeSymmetricBits(p.lPrime+p.epsilon+nHatBitLen,
p.prng)
if err != nil {
	return nil, errs.Wrap(err).WithMessage("could not sample delta")
}
mu, err := intSampleRangeSymmetricBits(p.lPrime+nHatBitLen, p.prng)
if err != nil {
	return nil, errs.Wrap(err).WithMessage("could not sample mu")
}
```

Figure B.7: The implementation uses `lPrime` instead of `l` as prescribed in the paper.
(pkg/proofs/cggmp21/affg/protocol.go#L443–L450)

- **Red Alert message validation does not check maps for spurious entries.** The `Validate` function just checks whether the Red Alert message has non-nil maps and whether the maps contain entries for all participating parties. It does not verify that these maps contain exactly the expected entries. This does not affect security, but the messages are incorporated in the transcript at a later stage, so we recommend ensuring that these messages are canonical.

```
// Validate checks a red-alert broadcast against the signing state and
sender's public parameters.
func (a *RedAlertBroadcast[P, B, S]) Validate(p *RedAlertParticipant[P, B, S],
sender sharing.ID) error {
	if a == nil {
		return cggmp21.ErrNil.WithMessage("red alert broadcast")
	}
	if a.BigD == nil || a.BigF == nil || a.PhiJ == nil {
		return cggmp21.ErrNil.WithMessage("red alert broadcast maps")
	}
	if len(a.Phi) == 0 {
		return cggmp21.ErrNil.WithMessage("red alert dec proof")
	}
```

Figure B.8: Validation includes checking that the maps are not `nil`.
(pkg/mpc/signatures/ecdsa/cggmp21/signing/messages.go#L195–L205)

```
for recipient := range signer.ctx.AllPartiesOrdered() {
	if recipient == sender {
		continue
	}
	d, ok := a.BigD[recipient]
```

```
	if !ok {
		return network.ErrMissing.WithMessage("red alert D for recipient
%d", recipient)
	}
	f, ok := a.BigF[recipient]
	if !ok {
		return network.ErrMissing.WithMessage("red alert F for recipient
%d", recipient)
	}
	proof, ok := a.PhiJ[recipient]
	if !ok || len(proof) == 0 {
		return network.ErrMissing.WithMessage("red alert aff-g* proof
for recipient %d", recipient)
	}
	recipientPaillierPublicKey, err := paillierPublicKeyFor(signer,
recipient)
	if err != nil {
		return err
	}
	if err := validatePaillierCiphertext(recipientPaillierPublicKey, d,
"red alert BigD"); err != nil {
		return err
	}
	if err := validatePaillierCiphertext(senderPaillierPublicKey, f, "red
alert BigF"); err != nil {
		return err
	}
}
```

Figure B.9: The validation includes checking that the map contains all recipients. (pkg/mpc/signatures/ecdsa/cggmp21/signing/messages.go#L211–L237)

```
for id := range p.cosigner().ctx.AllPartiesOrdered() {
	data, err := serde.MarshalCBOR(messages[id])
	if err != nil {
		return errs.Wrap(err).WithTag(base.IdentifiableAbortPartyIDTag,
id).WithMessage("cannot serialise red alert broadcast from %d", id)
	}

p.cosigner().ctx.Transcript().AppendBytes(redAlertBroadcastTranscriptLabel,
id.Bytes(), data)
}
```

Figure B.10: The messages are included in the transcript. (pkg/mpc/signatures/ecdsa/cggmp21/signing/redalert.go#L256–L262)

- **Runner message is caller-mutable.** When constructing a runner, the message is not copied, which means that the caller can mutate it after construction, leading to unexpected behavior. We recommend copying the message to prevent later mutation.

```
// NewRunner constructs a CGGMP21 signing runner.
func NewRunner[P curves.Point[P, B, S], B algebra.PrimeFieldElement[B], S
algebra.PrimeFieldElement[S]](
	ctx *session.Context,
	suite *sigecdsa.Suite[P, B, S],
	shard *cggmp21.Shard[P, B, S],
	message []byte,
	prng io.Reader,
) (network.Runner[*SignResult[P, B, S]], error) {
	cosigner, err := NewCosigner(ctx, suite, shard, prng)
	if err != nil {
		return nil, errs.Wrap(err).WithMessage("cannot create cosigner")
	}
	return &signRunner[P, B, S]{
		cosigner: cosigner,
		message:  message,
	}, nil
}
```

Figure B.11: The runner construction uses the message without copying. (pkg/mpc/signatures/ecdsa/cggmp21/signing/runner.go#L50–L66)

- **Auxiliary info exposes mutable aliases.** The auxiliary info exposes mutable aliases, allowing a user to mutate the `refreshID` and contained public keys later. We recommend freezing the maps to prevent mutation.

```
info := &AuxInfo{
	paillierSecretKey:      paillierSecretKey,
	paillierPublicKeys:     maps.Clone(paillierPublicKeys),
	ringPedersenSecretKey:  ringPedersenSecretKey,
	ringPedersenPublicKeys: maps.Clone(ringPedersenPublicKeys),
	refreshID:              refreshID,
}
```

Figure B.12: When constructing the auxiliary info, the `refreshID` is not copied. (pkg/mpc/signatures/ecdsa/cggmp21/cggmp21.go#L64–L70)

```
// PaillierPublicKeys returns the Paillier public keys indexed by sharing ID.
func (info *AuxInfo) PaillierPublicKeys() map[sharing.ID]*paillier.PublicKey {
	if info == nil {
		return nil
	}
	return maps.Clone(info.paillierPublicKeys)
}
```

Figure B.13: While the map is cloned, the contained public keys can be mutated. (pkg/mpc/signatures/ecdsa/cggmp21/cggmp21.go#L82–L88)

```
// RingPedersenPublicKeys returns the ring-Pedersen public keys indexed by
sharing ID.
func (info *AuxInfo) RingPedersenPublicKeys()
```

```
map[sharing.ID]*intcom.CommitmentKey {
	if info == nil {
		return nil
	}
	return maps.Clone(info.ringPedersenPublicKeys)
}
```

Figure B.14: While the map is cloned, the contained public keys can be mutated. (pkg/mpc/signatures/ecdsa/cggmp21/cggmp21.go#L107–L113)

```
// RefreshID returns the refresh identifier bound into signing transcripts.
func (info *AuxInfo) RefreshID() []byte {
	return info.refreshID
}
```

Figure B.15: The `refreshID` can be mutated. (pkg/mpc/signatures/ecdsa/cggmp21/cggmp21.go#L144–L147)

- **Incorrect function names in documentation.** The CGGMP21 signing README lists `SignResult.PartialSignatureOnlineAggregator` and `NewOfflineAggregator`, but in the implementation they have been renamed to `PartialSignatureCosigningAggregator` and `NewNonCosigningAggregator`.

```
## Aggregation

`SignResult.PartialSignature` returns the local partial signature.
`SignResult.PartialSignatureOnlineAggregator` returns an aggregator bound to
the successful online session and its transcript-derived state.

`NewOfflineAggregator` constructs a stateless aggregator for already validated
partial signatures. It checks that all partial signatures use the same
$\Gamma$ and sums their $\sigma_i$ values.
```

Figure B.16: pkg/mpc/signatures/ecdsa/cggmp21/signing/README.md#L54–L58

- **Point validation does not cross-check against the used curve.** The point validation applied for CGGMP21 messages calls the validation function on the underlying point, without verifying that this point is on the curve used by the signing parameters. This is currently not an issue because deserialization enforces points on the right curve.

```
func validatePoint[P curves.Point[P, B, S], B algebra.PrimeFieldElement[B], S
algebra.PrimeFieldElement[S]](point P, name string, allowZero bool) error {
	if utils.IsNil(point) {
		return cggmp21.ErrNil.WithMessage("%s", name)
	}
	if (!allowZero && point.IsZero()) || !point.IsTorsionFree() {
		return cggmp21.ErrValidationFailed.WithMessage("invalid %s",
```

```
name)
        }
        return nil
}
```

Figure B.17:
pkg/mpc/signatures/ecdsa/cggmp21/signing/messages.go#L317–L325

C. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From October 15 to October 16, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Bron Labs team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In addition to the targeted fixes for most issues, we reviewed PR #287, which implements a completely new prime generation routine based on a different paper to address TOB-BLCG-4. We fully reviewed the new implementation and compared it to the referenced paper. This review did not surface any findings that impact security.

In summary, Bron Labs has resolved all nine issues identified in the report. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Deserialization of Paillier elements can panic due to nil dereference	Low	Resolved
2	Round 1 DKG commitment message does not separate Pedersen values	Informational	Resolved
3	Deserialization failure on message receive does not include participant ID in error	Low	Resolved
4	Joye and Paillier safe prime generation reduces entropy of resulting safe primes	Informational	Resolved
5	Modular integer element size reports bits instead of bytes	Informational	Resolved
6	User is not sufficiently warned about thread-safe PRNG requirements for prime generation	Undetermined	Resolved
7	CGGMP21 signing proofs are not bound to their proof slot	Informational	Resolved

8	Discrete logarithm with ElGamal commitment proof verification does not bind witness to commitment	Undetermined	Resolved
9	Red Alert rounds lack round-state checks	Informational	Resolved

Detailed Fix Review Results

TOB-BLCG-1: Deserialization of Paillier elements can panic due to nil dereference
Resolved in commits `fceced9d` and `bd22af0f`. The first adds checks for the fields described in the finding, whereas the second adds additional checks and message validation for similar issues in other DKG messages.

TOB-BLCG-2: Round 1 DKG commitment message does not separate Pedersen values
Resolved in commit `eee4a701`. The commitment encoding now includes a separate length prefix for each Pedersen value.

TOB-BLCG-3: Deserialization failure on message receive does not include participant ID in error
Resolved in commit `8131f202`. Router and unicast decoding errors now carry the sender's participant ID.

TOB-BLCG-4: Joye and Paillier safe prime generation reduces entropy of resulting safe primes
Resolved in PR #287. The PR implements a completely new prime generation routine. The new routine is based on the generic coprime-candidate method in section 2 of Clavier, Feix, Thierry, and Paillier (PKC 2012): it independently samples each small-prime residue, reconstructs the residue modulo four times the product of small primes via the CRT, and constructs a uniformly random value in the target interval corresponding to those residue classes. Unlike the paper's provable-prime Algorithms 3.1 and 3.2, it samples fresh residues for every candidate and uses BPSW plus Miller–Rabin tests rather than a certified prime chain. For safe-prime generation, the CRT sieve additionally rejects residues that are one modulo a small prime.

TOB-BLCG-5: Modular integer element size reports bits instead of bytes
Resolved in commit `9b8f7b98` and commit `bd22af0f`. The implementation now computes the byte size correctly for all element bit sizes.

TOB-BLCG-6: User is not sufficiently warned about thread-safe PRNG requirements for prime generation
Resolved in commit `54134ef3`. The DKG `NewParticipant` documentation now states that the caller-supplied PRNG must be cryptographically secure and thread-safe.

TOB-BLCG-7: CGGMP21 signing proofs are not bound to their proof slot
Resolved in commit 177d92b1. Distinct transcript labels now bind each `enc-elg` and `aff-g` proof to its intended slot.

TOB-BLCG-8: Discrete logarithm with ElGamal commitment proof verification does not bind witness to commitment
Resolved in commit 3975446e. The proof now uses a single shared witness for the ElGamal plaintext and public point, so verification enforces that both are derived from the same discrete logarithm.

TOB-BLCG-9: Red Alert rounds lack round-state checks
Resolved in commit 0beb4c01. Red Alert now checks and advances its round state, rejecting out-of-order or repeated calls before they can reach the reported panic paths.

D. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Bron Labs under the terms of the project statement of work and has been made public at Bron Labs' request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.